# Create data-driven apps faster with ADBC and AI

# Who?

**Nic Crane**

- Arrow PMC member
- Independent educator and consultant
- ADBC user

**Ian Cook**

- Arrow PMC member
- Co-founder and CEO of Columnar
- ADBC developer

# Housekeeping

💬 Questions in the chat, we'll pause and answer them

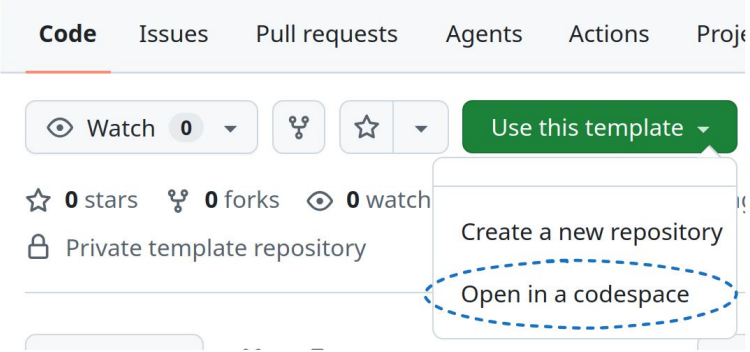
💻 Exercises can be found at
github.com/columnar-tech/adbc-lab

🎤 Q&A session at the end with Columnar crew

# Exercise Setup

## ☁️ Option 1: GitHub Codespaces

- Go to [github.com/columnar-tech/adbc-lab](https://github.com/columnar-tech/adbc-lab) and click ‘Use this template’ and then 'Open in a codespace'




## 🖥️ Option 2: Local setup

- Python 3.11+
- uv
- A running PostgreSQL instance

# Assumptions

- Data practitioner
- Comfortable with basic Python code
- Don’t know much about ADBC yet

# Polls

- What best describes your role?

- What's your primary language for data work?

- What data systems or databases do you use regularly?

**We build faster than ever.**

*But the data connectivity layer hasn't kept up.*

# By the end of this session, you'll be able to: 🎯

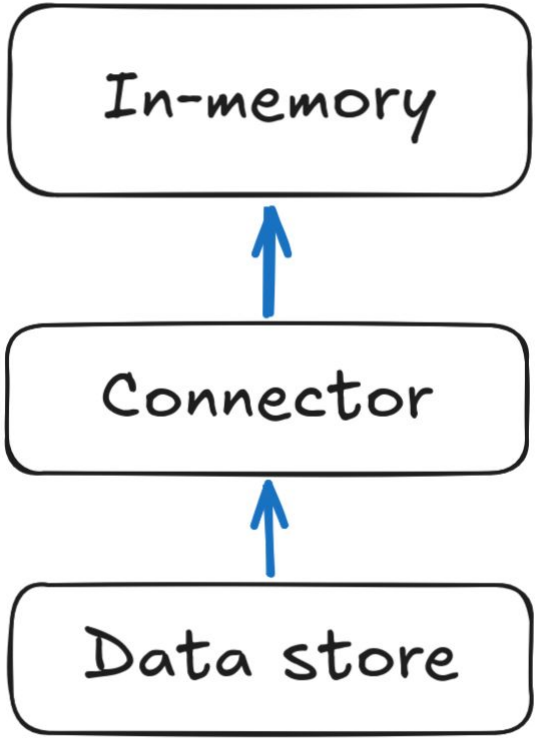
1. Understand why ADBC replaces slow connectors to speed up data pipelines, and how it impacts agentic analytics

2. Use ADBC and its surrounding tooling to install and work with database drivers without complicated setup

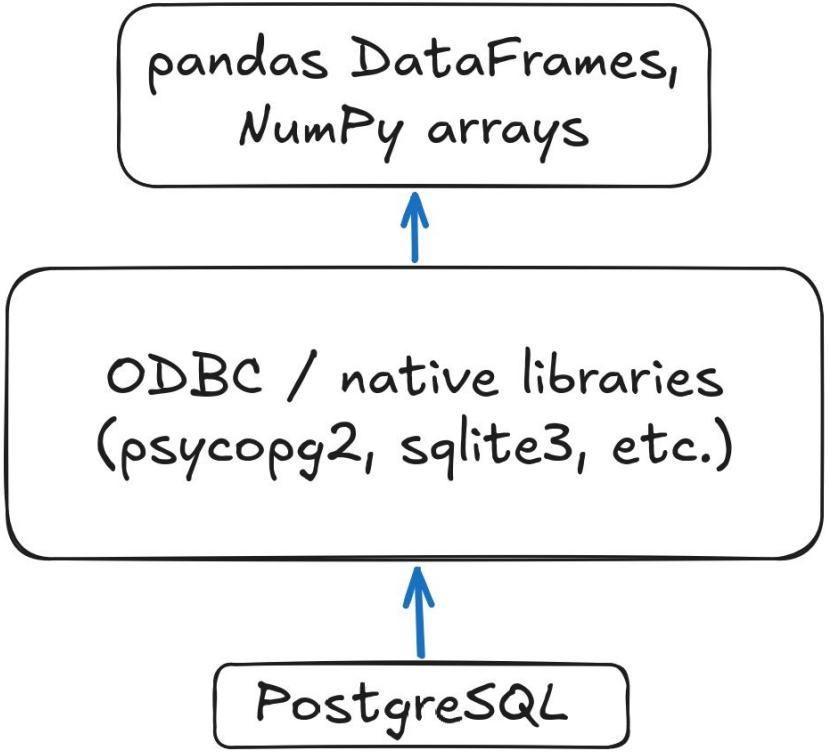
3. Switch between databases easily using a standardised interface

# The Data Connectivity Stack, and Where It's Falling Behind

# The data connectivity stack

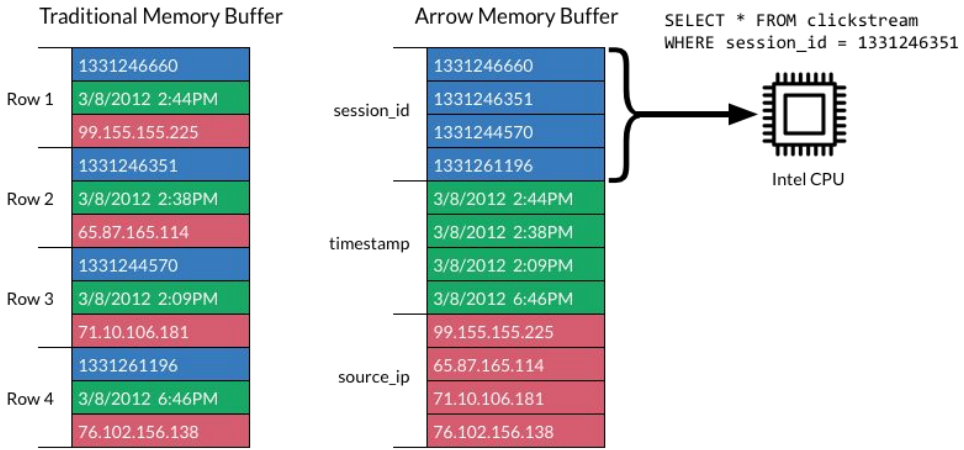


# The traditional setup



# Columnar data

| | session_id | timestamp | source_ip |
|---|---|---|---|
| Row 1 | 1331246660 | 3/8/2012 2:44PM | 99.155.155.225 |
| Row 2 | 1331246351 | 3/8/2012 2:38PM | 65.87.165.114 |
| Row 3 | 1331244570 | 3/8/2012 2:09PM | 71.10.106.181 |
| Row 4 | 1331261196 | 3/8/2012 6:46PM | 76.102.156.138 |



# Arrow is everywhere now 🏹

- Apache Arrow: the standard in-memory columnar format
  - Columnar
  - Standard data types and their in-memory layout

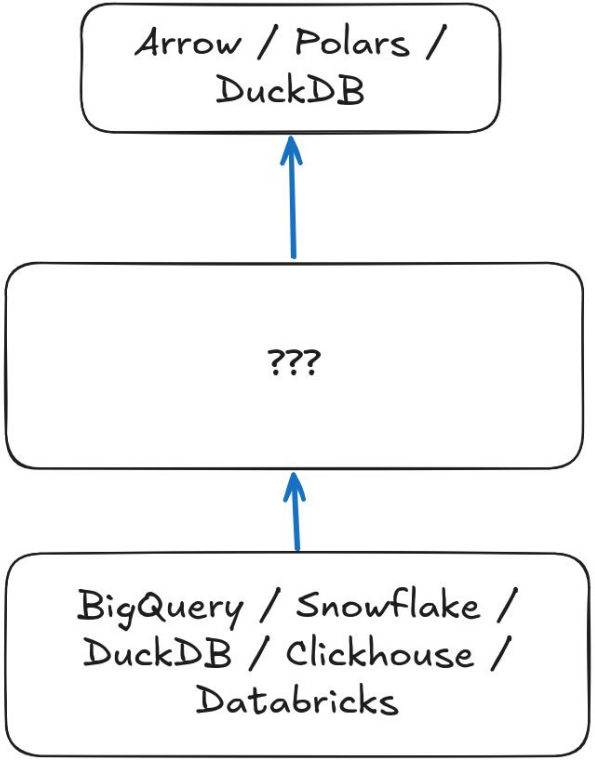
- Polars, DuckDB: built on or closely aligned with Arrow

- pandas 2.0+: Arrow-backed DataFrames available

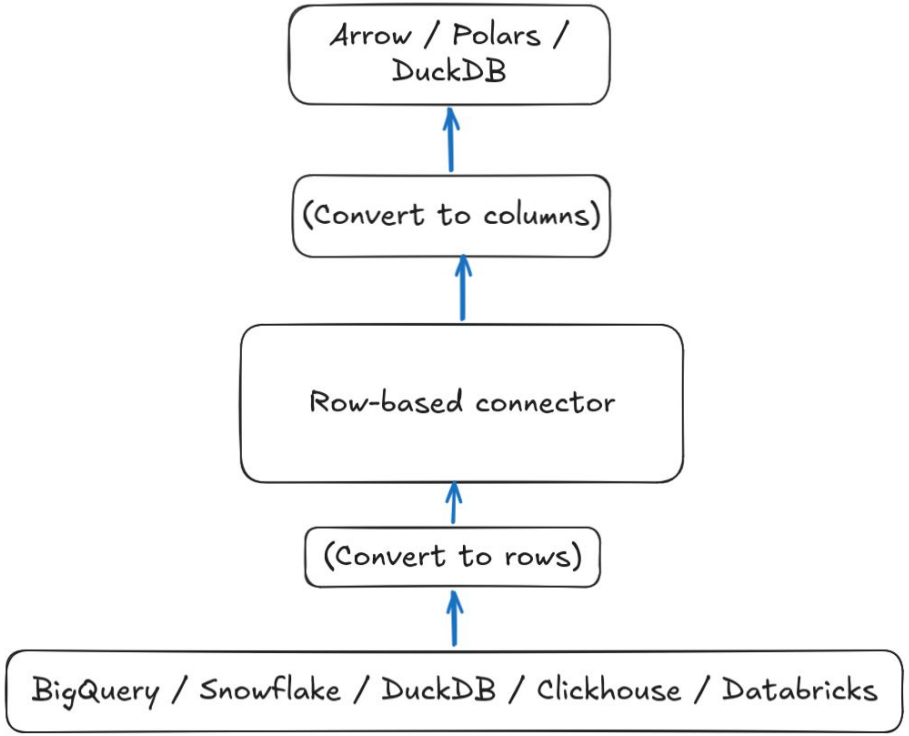
# Databases have moved too 🗄️

| Columnar | Row-oriented |
| --- | --- |
| BigQuery | MySQL |
| Clickhouse | Postgres |
| Databricks | SQLite |
| DuckDB | SQL Server |
| Snowflake | |

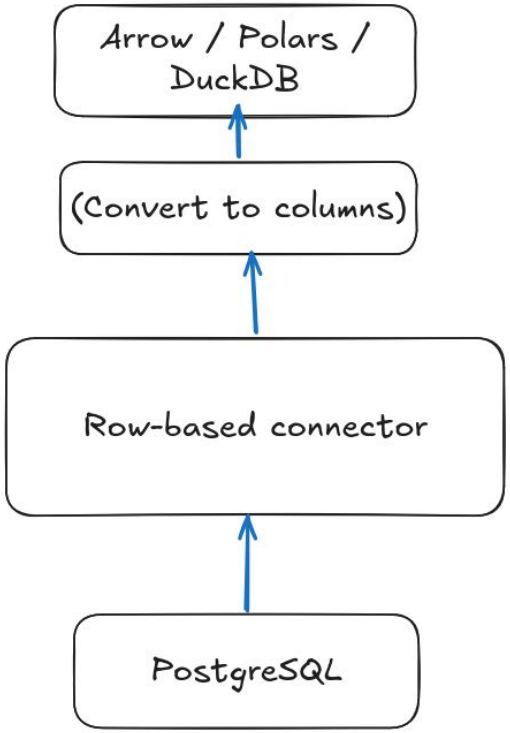
# The mismatch



# The mismatch - modern stack



# The mismatch - older stack



# Demo I: The Speed Difference

# Citi Bike trip data

- NYC bike share system, millions of trips

- Each record: start/end station, start/end time, user type, coordinates

- Loaded into a Postgres database

- ~1 million rows

Photo: Tom Marvel, CC BY 2.0

# Citi Bike Explorer

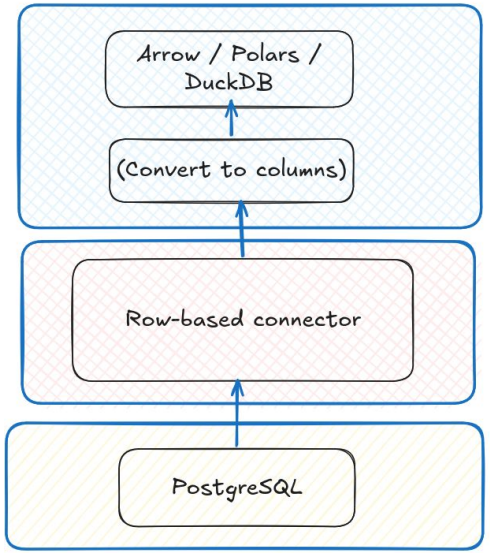
- Streamlit app

- Typical data analytics use case - working on data extract

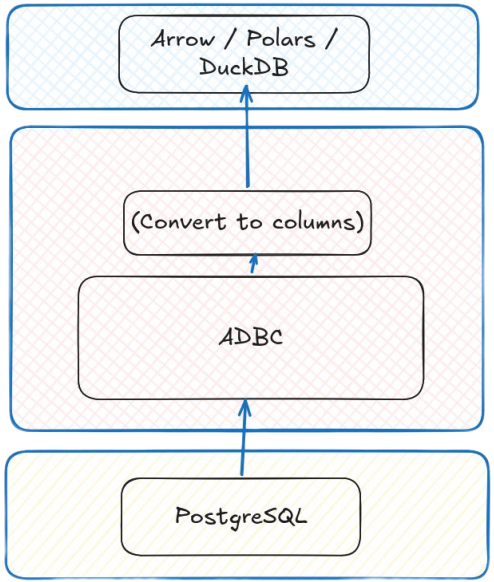
- Comparing connecting via row-based (psycopg2) and column-based connector (ADBC)

# Why was it faster? 🤔

Before

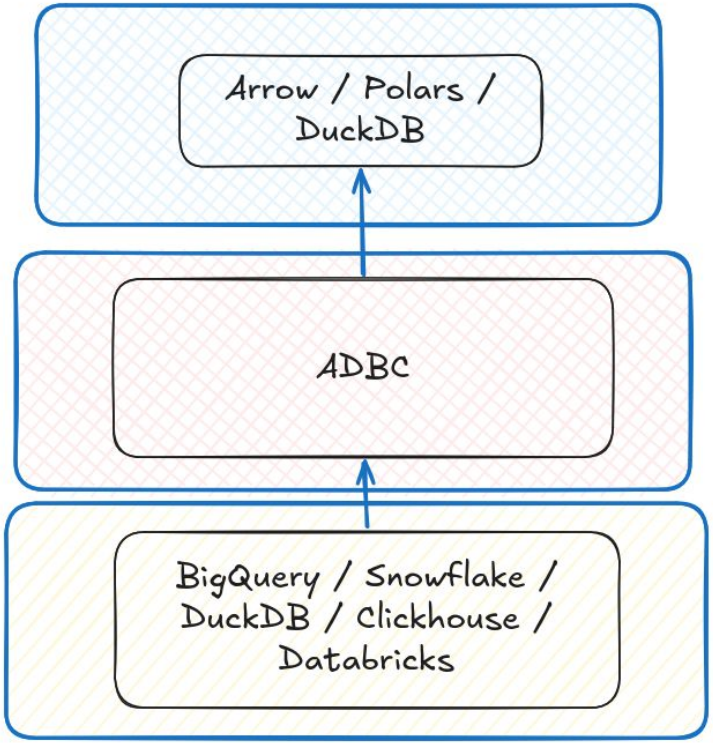



After




- ✅ Conversion in compiled code, not Python
- ✅ Arrow arrives natively, no rows touched
- ✅ No intermediate "bag of tuples" step
- ✅ ADBC driver optimised for performance

# When the database is columnar too 🚀




No intermediate conversion step

# DuckDB: ODBC vs ADBC 📊

TPC-H lineitem table, select and insert

No network overhead (client-side database)

**ADBC: 38x faster than ODBC 🔥**

Source: [duckdb.org/2023/08/04/adbc](duckdb.org/2023/08/04/adbc)

DuckDB

| Name | Time (s) |
|---|---|
| ODBC | 28.149 |
| ADBC | 0.724 |

# Not just ODBC

| Backend | ADBC | ODBC (turbodbc) | ODBC (arrow-odbc) | ODBC (pyodbc) | Native | Native Library |
|---|---|---|---|---|---|---|
| BigQuery | 1× | 2.69× | 2.68× | 4.17× | 1.73× | sqlalchemy-bigquery |
| DuckDB | 1× | driver crashes[1] | driver crashes[1] | 1750×+ [2] | 1× | duckdb-python |

Source: columnar.tech/blog/zero-copy-zero-contest

# Wide support across languages and databases 🌐

https://github.com/columnar-tech/adbc-quickstarts

# Questions translate to multiple queries

To answer the question:

***"Which stations are going to run out of bikes this weekend?"***,

we also need to run queries to answer:

*“Where are bikes flowing right now?”*

*“What happened on previous weekends?”*

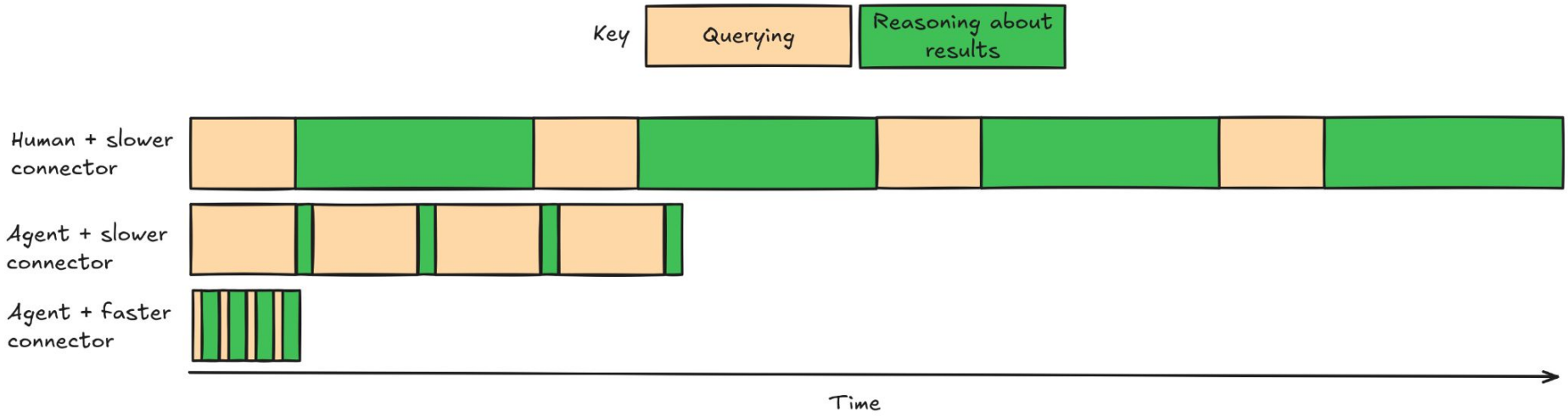
*“How do stations shift from weekday to weekend?”*

*“Is this week's lead-in worse than usual?”*

*“Which stations have high tourist exposure?”*

*“What are the values when we score and rank at-risk stations?”*

# Agentic analytics 🤖



# Getting Started with ADBC

# What is ADBC? 🏹

- Arrow Database Connectivity

- A modern standard for database connectivity

- Arrow-native: data arrives in the format your tools already use

- ADBC drivers are shared libraries - install once, use from any language

APACHE ARROW
ADBC

# Getting set up

1. Install the ADBC driver (database-specific component; pre-built shared library)

2. Install the driver manager (language-specific component; loads the driver and exposes it through familiar APIs)

# dbc 🛠️

CLI tool for installing ADBC drivers

One tool, works across all supported databases

dbc

# Demo II: Setting Up an ADBC Driver

# Basic dbc commands

| | |
|---|---|
| `uv tool install dbc` | *Install dbc using uv (other methods available)* |
| `dbc search` | *List all available drivers* |
| `dbc search -v` | *List all available drivers (verbose)* |
| `dbc info postgresql` | *Get information about a driver* |
| `dbc install postgresql` | *Install a driver* |

**dbc cheatsheet**: docs.columnar.tech/dbc/assets/cheatsheet.pdf

# ADBC Python Driver Manager

```
from adbc_driver_manager import dbapi
```

*Import the DBAPI-compatible interface*

```
dbapi.connect(driver="postgresql",
db_kwargs={"uri": uri})
```

*Connect to a database (returns a connection)*

```
conn.cursor()
```

*Create a cursor to run queries*

```
cur.execute("SELECT ... FROM trips")
```

*Execute a SQL query*

```
cur.fetch_arrow_table()
```

*Fetch results as an Arrow table*

# Exercise 1: Installing dbc & running your first ADBC query

Setup reminder: [github.com/columnar-tech/adbc-lab](https://github.com/columnar-tech/adbc-lab) and click ‘Use this template’ and then 'Open in a codespace'.

**1. Install dbc**

**2. Install an ADBC driver for Postgres**

**3. Connect and run a query**

Full instructions in exercise_1.txt

# Exercise 1: What you just learned

1. How to install the *dbc* cli tool to install ADBC drivers
2. How to find information about available drivers
3. How to install drivers for specific databases
4. How to use the ADBC driver manager in Python to connect to a database using ADBC

# How the pieces fit together ⚙️

Your code → Driver Manager → Driver → Database

```
from adbc_driver_manager import dbapi

dbapi.connect(driver="postgresql", ...)
```

# Portability

# Scenario: IT said to switch backends 💸

- Snowflake bill was 20K last month
- "Can't you just use <other database> for that workload?"
- Sure, but...

# Without ADBC: potentially complicated

- Check type mappings
  - Types can vary between connectors, e.g.
    - decimal → float
    - date → timestamp
    - Null handling
- Update any “glue” code for type conversion
- Driver configuration - ODBC headache or library-specific idiosyncrasies

Each library does it differently, so switching means new surprises

# With ADBC: change the driver name ✅

```
# Postgres
dbapi.connect(driver="postgresql", db_kwargs={"uri": "postgresql://..."})

# DuckDB
dbapi.connect(driver="duckdb", db_kwargs={"path": "my_data.duckdb"})

# Same code from here on
cursor = con.cursor()
cursor.execute("SELECT * FROM trips")
table = cursor.fetch_arrow_table()
```

# Demo III: Switching Backends

# Exercise 2: Switching backends in code

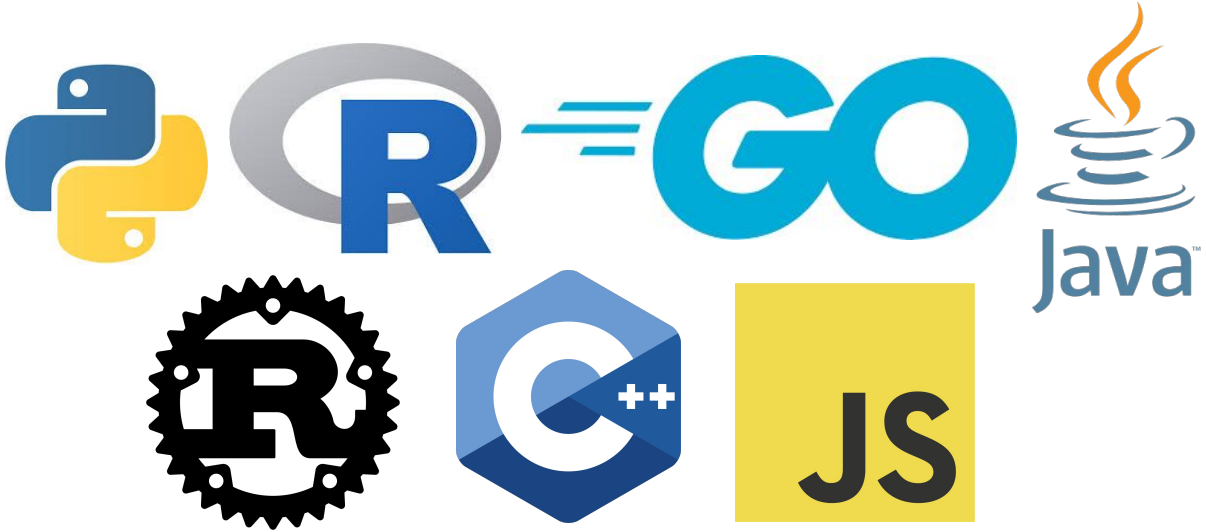
1. **Change the driver from Postgres to DuckDB in your code**
2. **Run the same query against a different backend**
3. **Compare the results**

Full instructions in exercise_2.txt

# Exercise 2: What you just learned

1. How to move data from 1 database to another using ADBC
2. How to run the same query against a different database

# Portability across languages too




Your data connectivity knowledge transfers, not just your code

# One tool across all your databases 🔑

One consistent way to:

- Connect
- Discover tables and schemas
- Query

# Why this matters for AI coding 🤖

Native libraries: agent needs psycopg2 API + duckdb API + type converters + ...

ADBC: one API, one set of types, works across backends

Less that can go wrong in generated code

# Agents connecting to databases without guidance 🤷

🎭 Hallucinate driver names that don't exist

🎯 Guess at connection URI formats

📚 Default to row-based connectors or native libraries from training data

# Agent skills for ADBC and dbc 🧠

Teaches AI coding tools how to use dbc and ADBC properly

Connect → discover schemas → query → switch backends

Install: `npx skills add columnar-tech/skills`

https://github.com/columnar-tech/skills

# Demo IV: The ADBC Skill

# So What Is ADBC and How Does It Work?

# ADBC is a standard, like Arrow 📐

| | Arrow | ADBC |
|---|---|---|
| **Standardises** | How data is represented in memory | How you connect and get data in and out |
| **Spec first** | ✅ | ✅ |

# Definitions

**ADBC** - the specification; C API is canonical implementation

**ADBC Driver** - individual library to interact with a database

**ADBC Driver Manager** - link to this for easier changing of backends

**dbc** - command line tool for installing ADBC drivers with less effort

# What ADBC is and isn't

✅ Cross-platform, cross-language, vendor-neutral, Arrow-native, open

❌ Not a backend, not a wire protocol, not a SQL transpiler, not a replacement for ODBC/JDBC for row-based workloads

# How ADBC relates to: 🧩

| Technology | Relationship |
| --- | --- |
| JDBC/ODBC | Same idea (universal standard), different era. ADBC is the columnar equivalent. |
| Arrow | Arrow = the data format. ADBC = how to get it in and out of databases. |
| Flight SQL | A wire protocol. ADBC can use it under the hood, but ADBC is the API you code against. |

# Where ADBC Fits in Your Workflows

# Where ADBC has the most impact ✅

📊 Analytical workloads pulling bulk data

🌍 Multi-language environments needing type consistency

🤖 Agent-driven workflows where simplicity and speed compound

🔄 Anywhere you're converting columnar → rows → columnar

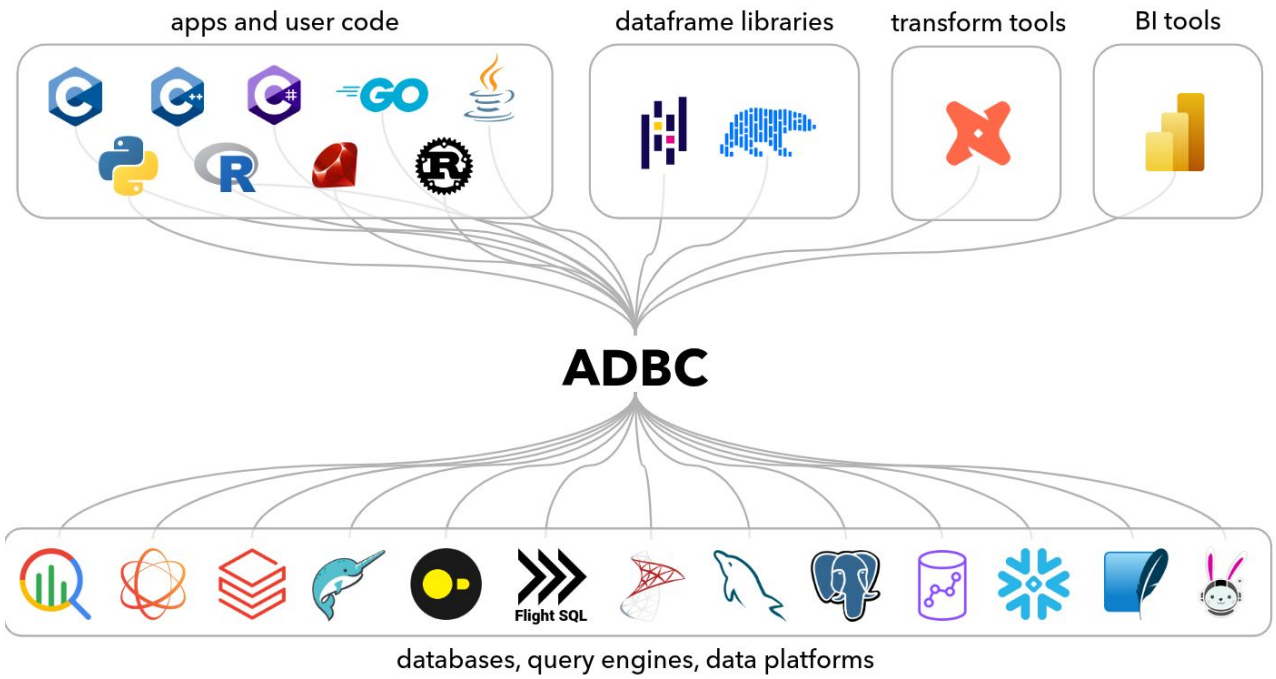
Example: Arrow tables straight into XGBoost, no pandas step (thefulldatastack.substack.com/p/training-xgboost-ducklake-adbc)

# When not to use it ⛔

OLTP / row-based transactional workloads (single-row inserts and updates)

ADBC is for bulk columnar access.

# You might already be using ADBC



# Where to go next 🚀

📖 **Columnar Cookbook** - cookbook.columnar.tech

💻 **adbc-quickstarts** github.com/columnar-tech/adbc-quickstarts

🛠️ **dbc** - columnar.tech/dbc

📝 **Columnar blog** - columnar.tech/blog

📚 **ADBC docs** - arrow.apache.org/adbc

💬 **Columnar Community Slack** - columnar-community.slack.com

# ADBC Latest news

ADBC 23 release highlights:

- **ADBC connection profiles**: Now you can store database connection parameters outside your code in a TOML file

- **A new JavaScript/TypeScript driver manager**: Now you can use ADBC drivers in Node.js apps

# What's ahead for ADBC 🔮

🟢 **Near-term** - Driver Foundry migration (drivers into adbc-drivers), expanded Java support via JNI

🟡 **Mid-term** - ADBC 1.2: more catalog metadata, warnings, requesting specific data types on read

🔵 **Long-term** - Native async APIs for Rust, JavaScript, and other async-first languages

# Q&A with Columnar Crew 🎤

Also: please fill in the post-workshop survey!